Transaction Management

Transactional Frameworks for Middleware Platform

Middleware Architecture (SCS 2314)

---

1. What is a Transaction?

- A transaction = change from one **stable state** to another **stable state**
- In between = **unstable intermediate states**
- Once complete → new stable state reached

```
{Initial Stable State}
         |
   [a(1), a(2)... a(n)]   <- sequence of actions
         |
{Final Stable State}
```

Example — Bank Transfer:

```
Before:  A = 1000, B = 500   <- stable
During:  A = 500,  B = 500   <- unstable (money in air)
After:   A = 500,  B = 1000  <- stable  ■
```

2. Atomic Action

- Atomic = **indivisible** — all or nothing
- Two atomic operations A and B = **mutually exclusive on dependency**
- They don't interfere with each other's outcome

```
Three valid execution orders:
1. Concurrently:  A || B
2. A first:       A --> B
3. B first:       B --> A
All three -> correct outcome ■
```

3. Composite Action

Serialised

- A **sequence** of atomic operations one after another
- Starts with initial state → ends with final state → within finite time

```
{Initial State}
      |
    a(1)
      |
    a(2)
      |
    a(n)
      |
{Final State}
```

Concurrent

- Multiple composite actions running **at the same time**
- They access a **shared object** → risky ■

```
Process A:  a(1) --> a(2) --> a(n)
                        \
                      [Shared Object]  <- both access this!
                        /
Process B:  b(1) --> b(2) --> b(n)
```

- If not managed → **data corruption** ■
- This is why we need **concurrency control**

4. ACID Properties

Every transaction must follow these 4 properties:

```
A -> Atomicity    (Failure Atomicity)
C -> Consistency  (Integrity)
I -> Isolation    (Concurrency Atomicity)
D -> Durability
```

A — Atomicity

- Transaction must complete **fully** or **not at all**
- Failure midway → **rollback** to original state

```
S1 --> [Action] --> S2    <- success ■
S1 --> [Action] --> FAIL  -> rollback -> S1 ■
Final state = EITHER S1 OR S2, never in between
```

C — Consistency

- Must go from one **consistent state** to another
- Intermediate inconsistent states = **hidden inside** transaction, never visible outside

```
Consistent --> [transaction] --> Consistent
                      |
               inconsistent states
               (hidden inside)
```

I — Isolation

- Multiple transactions run concurrently
- Result must equal as if they ran **one after another**

```
A || B  must equal either:
  -> A then B  (A ; B)
  OR
  -> B then A  (B ; A)
Intermediate states of one -> NOT visible to another ■
```

D — Durability

- Successful transaction = **Commit**
- After commit → changes are **permanent** even if system crashes ■

```
Transaction --> COMMIT --> permanent forever ■
Transaction --> FAIL   --> rollback, nothing saved ■
```

Managing ACID — 3 Mechanisms

ACID Property	Managed By
Atomicity	Rollback / Abort
Consistency	Rollback / Abort
Isolation	Concurrency Control (Locking)
Durability	Commit

5. Concurrency Control

Why Concurrency?

- Run multiple transactions simultaneously → **better performance**
- Full use of CPU, memory, resources
- Without it → transactions queue → slow ■

Goal

- Ensure **isolation** when transactions share data
- Main concept = **Serialisability**
- Main mechanism = **Locking**

```
Process A:  a(1) --> a(n)
                \
             [Shared Object] <- LOCK/RELEASE controls access
                /
Process B:  b(1) --> b(n)
```

Serialisability

- Schedule = **sequence of operations** from multiple transactions
- Serialisable = result equals a **serial schedule**

```
Serial:       T1 fully done --> then T2 starts   ■
Serialisable: T1 and T2 overlap BUT result = serial  ■
NOT Serial:   T1 and T2 overlap AND result different  ■
```

Serialisability Violations

Notation: Ri(x) = Transaction i reads x | Wi(x) = Transaction i writes x

Conflict = two transactions on same object, at least one writes

1. Unrepeatable Read:

```
Ti reads x --> Tj writes x --> Ti reads x again
                                <- DIFFERENT value ■
```

2. Dirty Read:

```
Ti writes x --> Tj reads x
                <- reads UNCOMMITTED value ■
                   if Ti rolls back -> Tj used wrong data ■
```

3. Lost Write:

```
Ti writes x --> Tj writes x
                <- Ti's write LOST ■
```

Schedules Example (S1, S2, S3)

S = the **order** in which operations from T1 and T2 actually execute. Same transactions, different timings = different schedules.

```
S1 (serialisable ■)       S2 (serialisable ■)       S3 (NOT serialisable ■)
■■■■■■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
T1: a = a - x             T1: a = a - x             T1: a = a - x
T1: b = b + x             T2: a = (1+r)a            T2: a = (1+r)a
T1: commit                T2: b = (1+r)b            T2: b = (1+r)b <- written
T2: a = (1+r)a            T2: commit                T2: commit
T2: b = (1+r)b            T1: b = b + x             T1: b = b + x  <- LOST! ■
T2: commit                T1: commit                T1: commit
```

S1 -> serial order -> serialisable ■
S2 -> different order but same result as S1 -> serialisable ■
S3 -> write-write conflict on b -> T1's write lost -> NOT serialisable ■

KEY RULE: Schedule is serialisable <-> dependency graph is ACYCLIC
          S3 has a cycle -> NOT serialisable ■

6. Locking

- Mechanism to **delay** a transaction's progress
- Enforces serialisability
- Most frequently used technique

Two Types of Locks

Lock Type	Used For	Who can hold it
Shared	Read only	Multiple transactions ■
Exclusive	Read + Write	ONE transaction only ■

Lock Commands

```
lock_exclusive(o)  -> acquire exclusive lock on object o
lock_shared(o)     -> acquire shared lock on object o
unlock(o)          -> release lock on object o
```

Rules

- Acquire lock **before** using shared object
- Shared lock → read only | Exclusive lock → read + write
- Cannot lock already locked object (except upgrade shared → exclusive)
- Must release **all locks** after commit or abort

```
BEFORE access --> acquire lock
DURING access --> lock held
AFTER commit  --> release lock
```

7. Two Phase Locking (2PL)

2PL Rule: After releasing a lock → cannot acquire any NEW locks

```
Phase 1 (Growing):    acquire locks
                             |
                   [Maximum Locking Point]
                             |
Phase 2 (Shrinking):  release locks
```

2PL Compliance Example

```
T1a NOT 2PL ■            T1b 2PL ■              T2 2PL ■
■■■■■■■■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■■■■    ■■■■■■■■■■■■■■■■■■■■■■■
lock_exclusive(a)         lock_exclusive(a)       lock_exclusive(a)
a = a - x                 a = a - x               a = (1+r)a
unlock(a) <- releases     lock_exclusive(b)       lock_exclusive(b)
lock_exclusive(b) <- NEW unlock(a)               b = (1+r)b
b = b + x                 b = b + x               unlock(b)
unlock(b)                 unlock(b)               unlock(a)
commit                    commit                  commit

T1a -> acquires new lock AFTER releasing -> violates 2PL ■
T1b -> acquires all locks BEFORE releasing -> 2PL ■
T2  -> acquires all locks BEFORE releasing -> 2PL ■
```

Schedule Sa vs Sb — Why it Matters

```
Sa (T1a + T2) ■                    Sb (T1b + T2) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
T1a: lock(a)                        T1b: lock(a)
T1a: a = a - x                      T1b: a = a - x
T1a: unlock(a)                      T1b: lock(b)      <- locks b immediately
T2:  lock(a)   <- grabs a!          T1b: unlock(a)
T2:  a=(1+r)a                       T2:  lock(a)      <- must WAIT
T2:  lock(b)   <- grabs b!          T1b: b = b + x    <- transfer done first
T2:  b=(1+r)b  <- WRONG! ■         T1b: unlock(b)
T2:  unlock(b)                      T2:  gets locks   <- now proceeds ■
T1a: lock(b)                        T2:  a=(1+r)a     <- correct value ■
T1a: b = b + x <- too late!         T2:  b=(1+r)b     <- correct value ■

Sa -> T2 calculated interest BEFORE transfer arrived -> wrong ■
Sb -> T2 waited -> transfer completed first -> correct ■
```

8. Deadlocks

- Deadlock = two transactions **waiting for each other** forever
- Neither can proceed → system stuck ■
- Called **circular wait**

```
T1 holds A, wants B --+
                      <-> circular wait ■
T2 holds B, wants A --+

Step by step:
T1: lock(A)       ■ gets A
T2: lock(B)       ■ gets B
T2: wants lock(A) ■ waiting... T1 has it
T1: wants lock(B) ■ waiting... T2 has it
-> DEADLOCK ■
```

Prevention

- Lock all objects in **same order** always

```
Both must lock: A first -> then B
T1: lock(A) --> lock(B) ■
T2: lock(A) --> lock(B) ■ (waits for T1 to release A)
-> No deadlock ■
```

Problems with prevention:

- Not always feasible ■
- Transactions often independent — don't know in advance what they'll need
- Reduces concurrency ■

Detection & Resolution

Detection — Wait-For Graph:

- Build graph of who is waiting for who
- Cycle in graph = deadlock

```
T1 --waits--> T2
^             |
+--waits------+
   CYCLE = DEADLOCK ■

In distributed systems -> nodes share info -> detect cycle locally
```

Resolution — Abort a Victim:

- Abort one transaction (victim) to break the cycle
- Choose victim to **minimise rollback cost**

```
Strategy 1 -> abort YOUNGEST transaction
Strategy 2 -> abort transaction LOCAL to detection site
After victim aborts -> other transaction proceeds ■
```

9. Transactional Middleware

How it Started

- Originally → transactions managed **inside DBMS** (Persistence Tier)

```
Process 1 --+
Process 2 --+--> [DBMS manages transactions]
Process n --+
```

Problem

- Systems became **distributed**
- Transactions applied to non-DB services too (messaging etc.)
- DBMS alone not enough ■ → move to **Business Logic Tier (Middleware)**

```
OLD: Processes --> DBMS handles everything
NEW: Processes --> MIDDLEWARE manages --> DBMS stores
```

Two Key Components

Component	Role
Transaction Manager (TM)	Coordinates overall transaction across ALL resources
Resource Manager (RM)	Manages ONE local resource. Ensures ACID for that resource

```
          +--> [RM1 - Database]
[TM] -----+--> [RM2 - Message Queue]
          +--> [RM3 - Legacy System]
```

10. JEE Architecture

- JEE = Java Enterprise Edition
- Platform for large scale distributed applications
- Organised into **3 Tiers**

```
Tier 1          Tier 2                      Tier 3
--------        ----------------------      ----------
[Client]  <--> [J2EE Server          ] <--> [Database]
               |- EJB Container              Server
               |   +- Enterprise Beans
               +- Web Container
                   |- JSP Files
                   +- Servlets

Tier 1 -> Client (browser, app)
Tier 2 -> Business Logic (J2EE Server)
Tier 3 -> Data Storage (Database)
```

11. EJB Container

- Enterprise Beans run inside **EJB Container**
- Container = runtime environment that **controls the beans**
- Provides all system-level services automatically

Service	What it does

Transaction Management	Handles ACID properties
Security	Controls access
Remote Client Connectivity	Allows remote clients to connect
Life Cycle Management	Creates/destroys beans as needed
Database Connection Pool	Manages DB connections efficiently

12. Types of EJBs

```
Enterprise Beans
      |
      +--> Session Beans   (temporary, per client)
      |
      +--> Entity Beans    (permanent, shared data)
```

Session Beans

- Represents a **client** in J2EE server
- Uses **Facade Pattern** → front face for client
- One bean per client only — dies when client disconnects → **non-persistent**

Property	Detail
One client	Serves ONE client only
Transient	Dies when client disconnects
Non-persistent	Data NOT saved to DB
Lifetime	Same as client session

Entity Beans

- Represents a **business object** in persistent storage
- Examples: Customer, Product, Order
- Multiple clients can access — data saved **permanently**

Property	Detail
Persistent	Data saved permanently
Shared	Multiple clients can access
Storage	DB, object store, file, legacy app
Represents	Real business domain object

Two types of persistence:

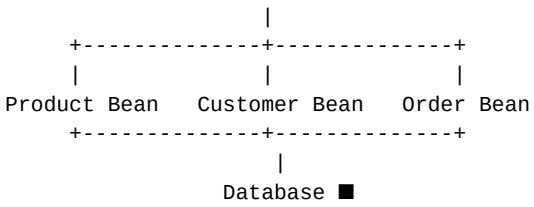
```
BMP (Bean Managed Persistence)
-> Developer writes SQL manually
-> More control, more work

CMP (Container Managed Persistence)
-> Container handles SQL automatically
-> Less code, easier ■
```

EJB Application Example

```
Tier 1       Tier 2                          Tier 3
---------    --------------------------------    ----------
[Browser] <-> [Servlet - Web Container    ]
              [Shopping Session Bean       ] <--> [Database]
              [ +-- Product Entity Bean   ]
              [ +-- Customer Entity Bean  ]
              [ +-- Order Entity Bean     ]

Browser --> Servlet --> Shopping Session Bean
```


Product Bean
Customer Bean
Order Bean
Database

FULL SUMMARY

Topic	Key Point
Transaction	Change between two stable states
Atomic Action	Indivisible — all or nothing
Composite Serialised	Sequence of atomic actions
Composite Concurrent	Multiple actions sharing one object
ACID	4 properties every transaction must have
Atomicity	All or nothing — rollback on failure
Consistency	Stable state -> stable state always
Isolation	Concurrent result = serial result
Durability	Commit = permanent, survives crashes
Concurrency Control	Ensures isolation via locking
Serialisability	Concurrent schedule = serial result
S1/S2/S3	Same transactions, different execution orders
Violations	Unrepeatable read, dirty read, lost write
Locking	Shared (read) or Exclusive (read+write)
2PL	Acquire ALL locks before releasing any
Sa vs Sb	Sa fails (non-2PL), Sb correct (2PL)
Deadlock	Circular wait — two transactions stuck
Prevention	Lock in same order always
Detection	Find cycle in wait-for graph
Resolution	Abort victim transaction
Transactional Middleware	TM coordinates, RMs manage local resources
JEE Architecture	3 tiers — Client, Business Logic, Database
EJB Container	Manages beans + provides all services
Session Bean	Per client — transient, non-persistent
Entity Bean	Shared data — persistent, permanent
BMP vs CMP	Manual SQL vs automatic SQL by container